

Chapitre 3

Les Objets du Navigateur en JavaScript

Sommaire

I.	Le modèle Objet	1
1.	Accès aux valeurs des propriétés	1
2.	Création d'instance d'objet : l'opérateur new et appel du constructeur	2
II.	Les objets prédéfinis JavaScript.....	2
1.	Les Objets du navigateur	2
2.	L'objet Document	7
3.	L'objet : form	10
4.	L'objet navigator.....	17
5.	L'Objet image	18

I. Le modèle Objet

Un objet est une collection de données appelées propriétés. Chaque propriété porte un nom servant à l'identifier. Par ailleurs, ces propriétés sont aussi des objets qui peuvent avoir eux-mêmes leurs propres propriétés.

La valeur d'une propriété peut être une chaîne de caractères, un nombre, un booléen ou, à son tour, un objet, y compris un tableau ou une fonction.

1. Accès aux valeurs des propriétés

a. Par la notation pointée (méthode classique)

Pour spécifier la propriété d'un objet, nous utilisons le nom de l'objet, suivi d'un point, suivi du nom de la propriété elle-même.

Quand la valeur d'une propriété est une fonction, nous disons que c'est une méthode. L'appel d'une méthode d'un objet doit être fait avec les parenthèses et les arguments s'ils existent.

Sa syntaxe : `Objet.propriété`

Exemples

Exemples de propriété : Pour un objet appelé `image` et possédant les propriétés `width` et `height`, nous faisons référence à ces propriétés comme suit : `image.width` `image.height`

Pour accéder à la propriété `prenom` d'un objet `individu` : `individu.prenom = "marcel";`

Exemple d'accès à une méthode : l'appel de la méthode `write` de l'objet `document` d'une page web est : `document.write('Bonjour');`

b. Par une matrice associative

Syntaxe : Sa syntaxe est la suivante : `objet ["propriété"] = "valeur";`

Exemple :

```
individu ["prenom"] = "marcel";
```

c. Par l'utilisation de `with`

Syntaxe : Sa syntaxe est la suivante :

```
with(objet) {  
    instructions  
}
```

Exemple :

```
with (individu) {  
    prenom = "marcel";
```

```
nom = "dupont";  
}
```

2. Création d'instance d'objet : l'opérateur new et appel du constructeur

Le terme **classe** signifie le type d'objet. On utilise un type d'objet à travers une instance d'objet qui est créée à l'aide de l'opérateur **new**

Syntaxe

La syntaxe générale de la création d'un objet est :

nomObjet = **new** typeObjet(paramètres); // typeObjet(paramètres) est la fonction **constructeur**. Nous pouvons créer autant d'instances d'objet que nous le voulons.

Exemple

laDate1=**new** Date(1966,2,20); // laDate1 est une instance de l'objet Date avec les paramètres 1966, 2, 20

Cette ligne de code construit un objet appelé laDate1 avec les trois propriétés : laDate1.an, laDate1.mois et laDate1.jour, qui prennent respectivement les valeurs 1966, 2 et 20.

L'ordre des arguments passés au constructeur est important.

L'objet laDate1 est dit aussi une instance de l'objet Date.

II. Les objets prédéfinis JavaScript

JavaScript est un langage à base d'objets. En effet, en JavaScript, quasiment tout est avant tout objet.

JavaScript est un langage objet et événementiel. Le développeur peut créer des objets sur la page, avec des propriétés et des méthodes et leur associer des actions en fonction d'événements déclenchés par le visiteur (passage de souris, clic, saisie clavier, etc...)

JavaScript intègre des objets prédéfinis. De plus, il offre au programmeur la chance de créer ses propres objets.

Les objets prédéfinis sont de deux catégories :

- **Les objets du navigateur** servant à fonctionner avec les éléments du document HTML en cours.
- **Les objets du noyau** sont utilisés pour la programmation des fonctions.

1. Les Objets du navigateur

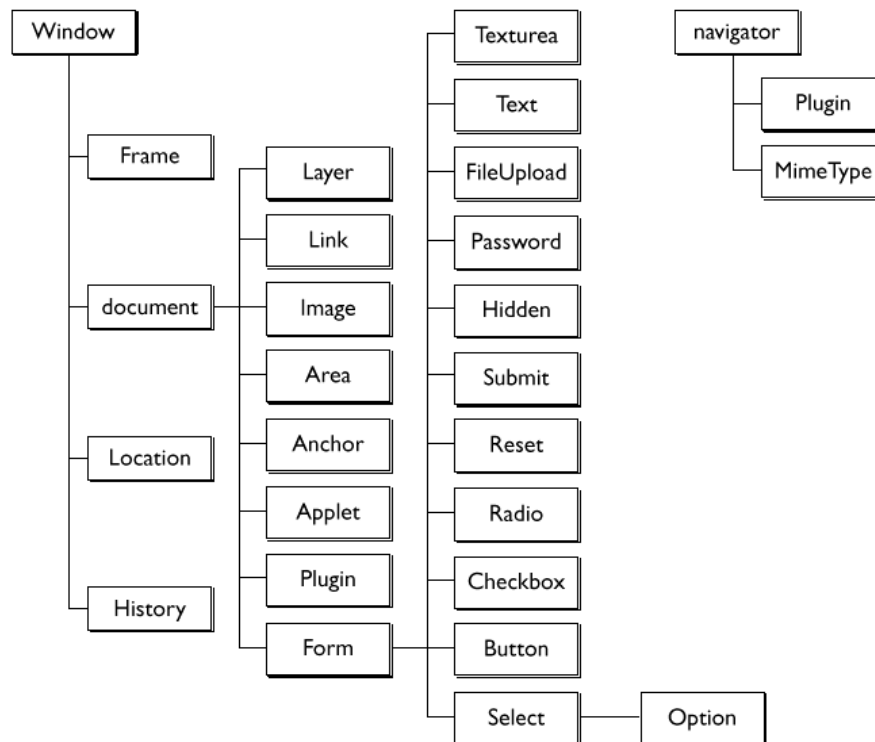
JavaScript nous offre un certain nombre d'objets. Ces objets fournissent des informations sur la page web courante, sur son contenu, ainsi que sur la session du navigateur; ils fournissent également des méthodes pour manipuler leurs propriétés. Par exemple, la méthode write() est liée à l'objet document :

```
document.write("Ceci est JavaScript!")
```

Lorsque vous ouvrez une page Web, le navigateur crée des objets prédéfinis correspondant à la page Web, à l'état du navigateur, et peuvent donner beaucoup d'informations qui vous seront utiles.

a. La Hiérarchie des objets du navigateur

La plupart des objets intégrés dans JavaScript font partie de la hiérarchie des objets du navigateur. Cette hiérarchie est construite à partir d'un objet de base unique, appelé Window



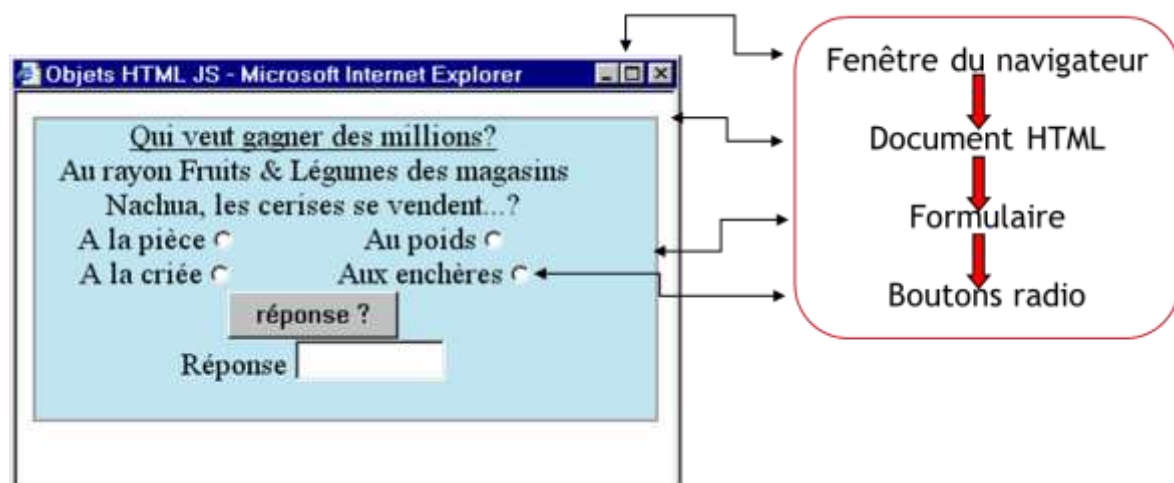
Les descendants d'un objet sont des propriétés de cet objet.

Cette hiérarchie, indispensable pour accéder à un objet donné.

Exemple

Pour accéder aux boutons radio dans la page web suivante il faut suivre la hiérarchie :

Fenêtre du navigateur, Document HTML, Formulaire, Boutons radio.



Le tableau suivant décrit les principaux objets du navigateur

Objet	Description
Navigator	L'objet navigator fournit des propriétés qui exposent aux scripts de JavaScript des informations sur le navigateur en cours.
Window	C'est l'objet où s'affiche la page, il fournit des méthodes et des propriétés pour s'occuper de la fenêtre actuelle du navigateur et tous les objets-enfants contenus dans celle-ci
Location	L'objet location fournit des propriétés et des méthodes pour s'occuper de l'URL en cours.
History	L'objet history contient une liste des adresses URL visitées par l'utilisateur et permet des interactions limitées avec cette liste, en fournissant des propriétés spécifiques telles que current, next et previous.
Document	L'objet document est l'un des objets les plus utilisés dans la hiérarchie. Il contient des objets, des propriétés et des méthodes pour fonctionner avec les éléments de document comprenant des formulaires (form), des liens (link), des points d'attache (anchor), et avec des applets.

Ces objets se trouvent dans chaque page HTML. De plus, et selon son contenu, un document pourrait les contenir. Ces objets sont largement dépendants du contenu de la page. En effet, mis à part des objets tels que Navigator, qui sont figés pour un utilisateur donné, le contenu des autres objets variera suivant le contenu de la page, car suivant la page les objets présents dans celles-ci (sous-objets des objets décrits précédemment) sont différents. Tenir d'autres objets.

Exemple

Un formulaire (form) défini par la balise <FORM> possède ses objets correspondants, tels que Button, Text, Select, etc.

i. Référencer les propriétés d'un objet

Lorsque nous parlons d'un objet en tant que propriété d'un autre objet, il sera noté en minuscule. Sinon, on écrit son premier caractère en majuscule pour signifier qu'on parle du type de l'objet. Cette notation n'est pas très importante, mais elle devient essentielle lors du codage.

Pour se référer à des propriétés spécifiques d'un objet, on doit indiquer le nom de l'objet et ses ancêtres, en se basant sur la hiérarchie des objets du navigateur.

Exemple

```
document.form1.text1.value
```

ii. Manipulation des objets avec l'instruction with

L'instruction with simplifie la manipulation des objets liés à une arborescence obligatoire.

Exemple

Toute manipulation de l'objet text1 doit être précédée par document.form1 et toute manipulation de form1 doit être précédée par document.

```
var resultat = document.form1.text1.value;
```

```
document.write("La somme est : "+ resultat);
```

On peut simplifier l'écriture de ces deux lignes en utilisant l'instruction with comme ceci :

```
with(document) {  
    var resultat = form1.text1.value;  
    write("La somme est : "+ resultat);  
}
```

On peut même placer l'objet document.form1 dans l'instruction with :

```
with(document.form1) {  
    var resultat = text1.value;  
    text1.blur(); // supprimer le focus de ce champ de texte  
}
```

b. L'objet window :

L'objet **window** est l'objet central de la hiérarchie et toutes les expressions JavaScript sont évaluées dans le contexte de cet objet. Pour accéder à une propriété de fenêtre, il n'est pas nécessaire de placer l'objet fenêtre devant son nom (window.nomPropriete); ceci s'applique aussi aux méthodes de Window, telle alert().

Parfois, il faut faire une **référence explicite** à l'objet fenêtre, entre autres lorsque nous voulons le passer comme argument d'une fonction par exemple. Dans ce cas, nous pouvons faire une référence à l'objet fenêtre par une autre propriété appelée **self** et qui donne le même résultat que Window.

La plupart des navigateurs permettent d'ouvrir plus d'une fenêtre à la fois. Ainsi, il doit y avoir plus qu'un objet fenêtre et, par conséquent, la référence implicite est une référence à l'objet fenêtre en cours, celui qui affiche le document HTML qui contient le code JavaScript qui est en exécution. Dans ce cas, si nous voulons accéder aux propriétés ou aux méthodes d'un autre objet fenêtre, nous devons utiliser une référence explicite à cet objet.

i. Propriétés de l'objet window

Propriétés	Description
window ou self	:fenêtre ouverte
Top	:fenêtre active
parent	:en présence de frames la fenêtre englobante
Opener	:fenêtre à l'origine de l'ouverture
name	:nom de la fenêtre
Length	:nombre de frame
defaultStatus	:statut initial de la barre d'état
Status	:valeur actuelle du statut
Location	:URL
Document	:contenu de la page WEB

ii. Méthodes :

Méthode	Description	Exemple
alert()	Affiche un texte avec une option OK.	<code>alert("Attention Erreur!")</code>
confirm()	Affiche un texte avec les options OK et Cancel. Si l'utilisateur choisi OK, la fonction retourne true, sinon elle retourne false.	<code>ret = confirm("Êtes-vous sûr?")</code>
prompt()	Affiche le texte de son premier argument et un texte de saisie affichant par défaut son deuxième argument lequel sera retourné une fois que l'utilisateur aura cliqué sur OK.	<code>nom = prompt("Entrer le nom", "Paul")</code>
open()	ouvre une nouvelle fenêtre.	
close()	ferme la fenêtre en cours.	
setTimeout ()	Fixe un délai en millisecondes avant l'exécution d'une instruction Javascript	<code>var ident = window.setTimeout("alert ('Délai écoulé')", 1000);</code>
clearTimeout ()	Stope un Timeout avant son execution	<code>window.clearTimeout (ident);</code>

Remarque concernant la méthode open() :

Les caractéristiques de la fenêtre doivent être séparées par des virgules. Sauf les deux dernières, les autres caractéristiques prennent des valeurs yes ou no, la valeur par défaut étant no comme indique le tableau suivant :

Paramètre	Description	Exemple
width	largeur de la fenêtre en pixel	width=300
height	hauteur de la fenêtre en pixel	height=400
top	position par rapport au haut de l'écran	top=200
left	position par rapport à gauche de l'écran	left=300
menubar	présence de la barre de menu	menubar=no
toolbar	présence de la barre d'outils	toolbar=1
scrollbars	présence des barres d'ascenseur	scrollbars=0
resizable	redimensionnement de la fenêtre	resizable=yes
fullscreen	passage en mode plein écran	fullscreen=no

Valeur en pixel
Etat yes/no
ou 0/1

2. L'objet Document

L'objet **Document** est hiérarchiquement placé sous l'objet **window**. Il permet de manipuler tous les objets qui sont inclus dans les documents HTML (images, textes, formulaires...). Il s'agit sans doute de l'objet le plus riche et le plus important, car dorénavant, vous aurez à votre disposition des outils qui permettent de changer de fond en comble le contenu et le design d'une page d'une manière dynamique.

a. Les propriétés de l'objet document

Propriété	Description
alinkColor	(Couleur des liens)
bgColor	(Couleur de fond de page)
characterSet	(Propriété contenant le type d'encodage du document)
cookie	(Chaîne contenant les cookies du domaine)
defaultCharset	(Jeu de caractères)
Domain	(Domaine de l'adresse de la page)
fgColor	(Couleur du texte)

fileSize	(Taille de la page en octets)
lastModified	(Date de dernière modification)
Location	(Url de la page)
mimeType	(Type de document)
Protocol	(Type de protocole de l'url)
referrer	(Adresse de la page d'origine)
Title	(Titre de la page)
URL	(Adresse de la page)
URLUnencoded	(Adresse codée de la page)
vlinkColor	Couleur des liens visités)

b. Les méthodes de l'objet *document*

Méthode	Description
document.write(' contenu ')	(Ecrit une chaîne de caractères dans le document)
Document.writeln('contenu')	(Ecrit du texte dans le document)
document.open()	ouvrir une page HTML (pop up).
document.getSelection()	donne le texte sélectionné

c. Les tableaux d'objets du navigateur

Certains objets du navigateur possèdent des propriétés dont les valeurs sont des tableaux. Ces tableaux sont utilisés pour stocker des données dont la quantité n'est pas connue d'avance

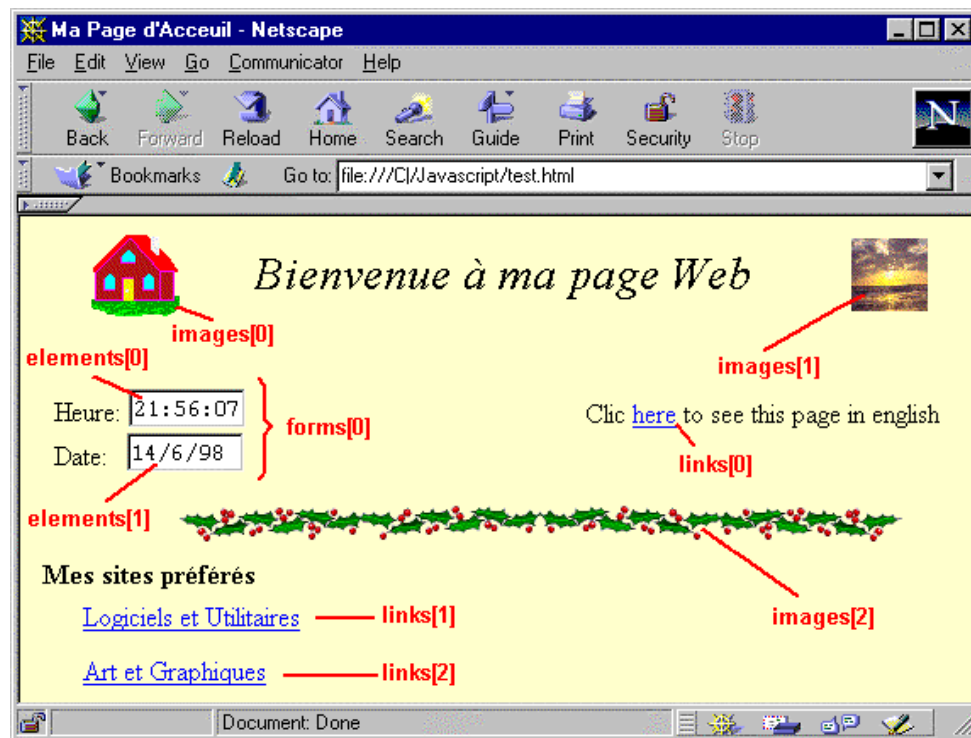
tableau	Description
document.images[]	permet de récupérer et modifier les images de votre page HTML
document.links[]	permet de récupérer et modifier les liens de votre page HTML.
document.forms[]	permet de récupérer et modifier les informations de votre formulaire.

Exemples de tableaux d'objets du navigateur.

Exemple 1 : Tableau d'images

Dans un document HTML, on peut chaque fois ajouter ou supprimer des images. De cette façon, le nombre d'images placées est indéterminé. Par conséquent, il est préférable de se référer à l'image par un indice plutôt que par son nom. JavaScript nous permet de le faire par un tableau appelé images. On dit aussi que images reflète l'objet créé par la balise dans le document HTML.

Exemple 2 : Des références, avec des tableaux, à des objets d'un document HTML.



Puisqu'un document HTML peut avoir plusieurs formulaires, les objets form sont stockés dans un tableau appelé forms. On dit aussi que forms reflète l'objet créé par la balise <FORM>. De même, les éléments d'un objet form, tels les champs de texte, les boutons, etc., sont stockés dans un tableau appelé elements.

Pour se référer au champ de texte Heure, on écrit :

```
document.forms[0].elements[0]
```

d. Tableaux d'objets du navigateur et éléments HTML

Le tableau suivant présente les principaux objets du navigateur et des éléments qu'ils reflètent dans le document HTML.

Tableaux d'Objets	Éléments reflétés
document.images[]	Les balises
document.forms[]	Les balises <FORM>
document.Links[]	Les balises <AREA HREF="...">, et les objets Link
document.applets[]	Les balises <APPLET>
document.Anchors[]	Les balises <A> contenant un attribut NAME
document.history[]	Les entrées history de l'objet window
document.forms[].elements[]	Les éléments d'un formulaire tels que Text, Button, Check box, etc.
document. forms[].select[]	options Les options d'un objet select spécifiées par la balise <OPTION>

3. L'objet : form

Avec l'objet form, qui se trouve sous l'objet document dans la hiérarchie JavaScript, on a accès aux formulaires définis dans un fichier HTML ainsi qu'à leurs éléments.

a. Utilisation de formulaires dans un document

Accès direct via l'index de chaque formulaire ou éléments

`document.forms[1].éléments[1].option[1].propriété`

Accès par la propriété "name" ou « id »

Il existe d'autres manières, plus courtes, et c'est généralement le cas, d'accéder à un formulaire et ses éléments. Ainsi, on peut utiliser l'attribut "name" ou l'attribut " id " .

Pour utiliser cette technique, on doit spécifier la propriété "name" lors de la déclaration du formulaire et des éléments

`<form name="maForm"...>` pour un formulaire

ou `<input name="monElement"...>`.

Une fois le formulaire bien identifié, on n'a pas à localiser sa position dans le tableau mais simplement à donner son nom ou son id . Si le nom et l'id sont indiqué dans la balise c'est l'id qui identifie l'objet.

`document.maForm.monElement.propriété`

b. Les propriétés de l'objet FORM

Propriétés	Description
Action ()	Définit l'URL où le formulaire sera envoyé
Elements	Tableau représentant les éléments du formulaire
Length	Nombre d'éléments à l'intérieur du formulaire –
Method	Définit le type d'envoi du formulaire (get ou post)
Name	Définit le nom du formulaire
Target	Définit la page (fenêtre ou frame) de réponse
Parent	Indique une fenêtre d'un cadre (frame)

c. Les éléments d'un formulaire

Les sous objets ELEMENTS de l'objet Form :

- **Bouton** : regroupe les balises `<button>` ainsi que les `<input>` de types reset et submit
- **Case**: regroupe les `<input>` de types checkbox et radio
- **Select** : avec le sous objet option
- **Textarea** : champs de texte sur plusieurs lignes
- **Text** : balise `<input>` de types text, password, hidden, number, tel, date, datetime, range, tel et file

d. Attributs (propriétés) des éléments d'un formulaire :

Tous les éléments de formulaire possèdent ces attributs :

Propriété	Description
form	le formulaire auquel appartient l'élément
name	nom de l'objet (le fameux nom qui nous sert à désigner cet objet en JS)
type	type de l'objet

Pour les attributs spécifiques, le tableau suivant résume quel attribut pour quel élément :

Élément	checked	defaultchecked	disabled	maxlength	readonly	size	value	defaultvalue
Bouton*	non	non	oui	non	non	non	oui	non
Case*	oui	oui	oui	non	non	non	oui	non
Hidden	non	non	non	non	non	non	oui	non
Select	non	non	non	non	non	oui	non	non
Texte*	non	non	oui	oui	oui	oui	oui	oui
Textarea	non	non	oui	non	oui	non	oui	oui

Le sous objet option de Select :

Les attributs de l'objet option

- **disabled**, **form** et **value**
- **index** : indice du choix parmi le tableau options de l'élément select
- **label** : désigne le groupe auquel appartient ce choix
- **selected** et **defaultselected** : vaut true si ce choix est sélectionné / sélectionné par défaut

e. Méthodes spécifiques aux formulaires

- **focus** : donne le focus à cet élément (pour une zone de texte, place le curseur à l'intérieur)
- **blur** : enlève le focus de cet élément (en quelque sorte le contraire de focus)
- **click** : simule un clic de souris sur cet élément
- **select** : sélectionne ("surligne") le texte de ce champ

f. Méthodes pour les éléments d'un formulaire

Le tableau suivant résume quelles méthodes pour quels éléments :

Élément	focus	blur	click	select
Bouton*	oui	oui	oui	non
Case*	oui	oui	oui	non
Select	oui	oui	non	non
Textarea	oui	oui	non	oui
Texte*	oui	oui	non	oui

g. Évènements spécifiques aux formulaires

Un événement est une réaction à une action émise par l'utilisateur, comme le clic sur un bouton ou la saisie d'un texte dans un formulaire.

Les Évènements spécifiques aux formulaires :

- **onFocus** : lorsque l'élément reçoit le focus (pour une zone de texte, quand on place le curseur à l'intérieur)
- **onBlur** : lorsque l'élément perd le focus (en quelque sorte le contraire de onFocus)
- **onChange** : lorsque la valeur / l'état de l'élément change (quand on coche la case, qu'on modifie le texte, etc.)
- **onSelect** : lorsqu'on sélectionne (quand on "surligne") le texte de ce champ

h. Évènements spécifiques aux éléments d'un formulaire :

Pour les éléments, le tableau suivant résume quels événements pour quels éléments :

Élément	onFocus	onBlur	onChange	onSelect
Bouton*	oui	oui	non	non
Case*	oui	oui	oui	non
Select	oui	oui	oui	non
Textarea	oui	oui	oui	oui
Texte*	oui	oui	oui	oui

i. Test des éléments d'un formulaire :

Exemple 1 : Accès aux zones de texte

Lire une valeur dans une zone de texte

Voici une zone de texte. Entrer une valeur et appuyer sur le bouton pour contrôler celle-ci.

Contrôler

Pour contrôler la zone de texte on propose le script et le document HTML suivants :

<HTML>

<HEAD>

<SCRIPT LANGUAGE="javascript">

```
function tester(MyForm) {
var test = document.MyForm.Zsaisi.value;
alert("Vous avez tapé : " + test);
}
```

</SCRIPT>

</HEAD>

<BODY>

```
<FORM NAME="MyForm">
<INPUT TYPE="text" NAME="Zsaisi" VALUE=""><BR>
<INPUT TYPE="button" NAME="bouton" VALUE="Contrôler" onClick="tester(MyForm)">
</FORM>
</BODY>
</HTML>
```

Exemple 2 : Accès aux cases à cocher

Soit le formulaire suivant comportant 3 checkbox et un bouton corriger . Il faut sélectionner les numéros 1,2 et 4 pour avoir la bonne réponse.

Entrez votre choix :

- ☐ Choix numéro 1
- ☐ Choix numéro 2
- ☐ Choix numéro 3
- ☐ Choix numéro 4

Corriger

Le document HTML complet qui réalise ce traitement est celui-ci :

```
<HTML>
<HEAD>
<script language="javascript">
function reponse(f) {
if ( (f.check1.checked) == true && (f.check2.checked) == true && (f.check3.che
cked) == false && (f.check4.checked) == true)
{ alert("C'est la bonne réponse! ") }
else
{alert("Désolé, continuez à chercher.")}
}
</SCRIPT>
</HEAD>
<BODY>
Entrez votre choix :
<FORM NAME="form">
<INPUT TYPE="CHECKBOX" NAME="check1" VALUE="1">Choix numéro 1<BR>
<INPUT TYPE="CHECKBOX" NAME="check2" VALUE="2">Choix numéro 2<BR>
<INPUT TYPE="CHECKBOX" NAME="check3" VALUE="3">Choix numéro 3<BR>
<INPUT TYPE="CHECKBOX" NAME="check4" VALUE="4">Choix numéro 4<BR>
<INPUT TYPE="button"NAME="but" VALUE="Corriger" onClick="reponse(form)">
</FORM>
</BODY>
</HTML>
```

Exemple 3.1 : Accès à des boutons radio

Réaliser le formulaire suivant et un script Js permettant d'indiquer si oui ou non une case à cocher est activée. Les valeurs possibles sont true ou 1 ou false ou 0. En effet si la case « avec cadres » est cochée un fichier « fichiercadre.html » sera localisé sinon c'est un fichier « fsansc.html » qui va être appelé.

☐ avec cadres ☐ sans cadres

Lancer

Le script est celui-ci :

```
<body>

<script>
function winsuiv() {
if(document.formulaire_test.rcadre[0].checked == true)
window.location.href="fichiercadre.htm"; else
if(document.formulaire_test.rcadre[1].checked == true)
window.location.href="fsansc.htm";
else alert("Veuillez faire un choix"); }
</script>
<form name="formulaire_test" action="">
<input type="radio" name="rcadre" value="avec"> avec cadres
<input type="radio" name="rcadre" value="sans"> sans cadres <br>
<input type="button" value="Lancer" onClick="winsuiv()">
</form>
</body>
```

Exemple 3.2 : Accès à des boutons radio

Soit le formulaire suivant qui, suite au clic sur le bouton, l'état des cases s'affiche :

On propose le document Html et le script suivants :

```
<head><meta charset="utf-8"></head>
<script type="text/javascript">
function afficherEtat(f) {
var s = "";
for (var i=0; i<f.elements["radiobutton"].length; i++) {
var btn = f.elements["radiobutton"][i];
s += btn.value + ": " + btn.checked + "<br>";
}
document.write(s);
}
</script>
<form>
<fieldset style="width:20%">
<legend style="text-align:center">Test RGB</legend>
```

```

<input type="radio" name="radiobutton" value="R" />Red
<input type="radio" name="radiobutton" value="G" />Green
<input type="radio" name="radiobutton" value="B" />Blue
</fieldset>
<br>
<input type="button" value="Afficher l'état" onclick="afficherEtat(this.form) ;"/>
</form>

```

Exemple 4 : Accès Liste de sélection

Soit le formulaire suivant représentant une liste de sélection de filières pour l'orientation au département TI de l'ISET . Le clic sur le bouton « vérifier » permet de vérifier si l'utilisateur a choisi une option ou pas.

Orientations Département TI



Le code correspondant au test est :

```

<script>
function selection(f) {
var index = f.elements["slist"].selectedIndex;
if (index == -1) {
window.alert("Aucune filère choisie");
}
else
{
var element = f.elements["slist"].options[index];
window.alert("L'Element " + index + " (c'est à dire la filière: " + element.text + ", value: " + element.value + ") choisie");
}
}
</script>
<form>
    <h1>Orientations Département TI</h1>
    <select name="slist" size="3">
    <option value="D">DSI</option>
    <option value="R">RSI</option>
    <option value="S">SEM</option>
    </select>
    <input type="button" value="Vérifier" onclick="selection(this.form);" />
</form>

```

j. La validation d'un formulaire : Avant l'envoi d'un formulaire

Globalement, la validation des champs par JavaScript, est potentiellement beaucoup plus problématique.

Meilleure méthode de validation

- ✓ Il faut d'abord utiliser les bons boutons :

Le/les bouton(s) pour envoyer le formulaire doivent être :

- soit de type « submit » : `<input type="submit" value="Ok" />` ;
- soit de type image, si on veut un bouton « submit » sous forme d'image : `<input type="image" src="ok.png" alt="ok" />`.

Remarque : Il est possible d'utiliser des feuilles de styles pour changer l'apparence des boutons : couleurs, bordures, image de fond, etc.

- ✓ Ensuite, le bon événement : `onsubmit()`

Cet événement est déclenché lorsque le formulaire est sur le point d'être envoyé, suite à l'appui sur la touche Entrée, ou d'un clic sur un bouton « submit », ou toute autre action indiquant au navigateur d'envoyer le formulaire.

Donc l'exécution d'un script devant vérifier le contenu du formulaire (ou faisant éventuellement d'autres traitements liés à l'envoi) doit toujours se faire sur l'événement "onsubmit" !

Il faut donc créer et appeler une fonction `valider()` sur le "onsubmit", et celle-ci devra renvoyer un booléen pour indiquer si le formulaire doit être envoyé ou non .

Ce qui donne donc :

```
<form action="" onsubmit="return valider(this)" method="" name="formSaisie">
  <p>
    <label for="prenom">Saisissez votre prénom :</label>
    <input type="text" id="pren" />
    <input type="submit" value="Ok" />
  </p>
</form>
<script>
function valider(frm){
  if(frm.elements['pren'].value != "") {
    return true;
  }
  else {
    alert("Saisissez le prénom");
    return false;
  }
}
</script>
```

Ainsi, que l'on appuie sur la touche Entrée ou que l'on clique sur le bouton « submit », la fonction `valider()` sera toujours appelée.

4. L'objet navigator

L'objet *navigator* est un objet qui permet de récupérer des informations sur le navigateur qu'utilise le visiteur. Toutes les **propriétés** de l'objet *navigator* sont en **lecture** seule, elles servent uniquement à récupérer des informations et non à les modifier (il serait idiot de vouloir modifier les informations d'un navigateur...).

i. Les propriétés de l'objet navigator

ces propriétés permettent en fait de retourner des portions de l'information sur votre navigateur. Etant donné que ces propriétés sont statiques, il est nécessaire de les faire précéder par *navigator*

Propriété	Description	Exemple
<i>appCodeName</i>	retourne le code du navigateur. Un navigateur a généralement pour nom de code <i>Mozilla</i> , le moteur utilisé par la plupart des navigateurs (internet explorer, netscape, mais aussi beaucoup de navigateurs sous Unix...). Cette valeur sera différente si le navigateur du client n'est pas basé sur un autre moteur (e.g. Opera, ...).	Mozilla
<i>appName</i>	retourne le nom du navigateur (la plupart du temps la marque). Cette propriété est utile pour différencier les navigateurs de Netscape et de Microsoft).	Google Chrome
<i>appVersion</i>	retourne la version du navigateur. Cette propriété prend la forme suivante : Numéro de version(plateforme (système d'exploitation), nationalité) Elle est utile pour connaître le système d'exploitation de l'utilisateur, mais surtout, associée avec la propriété <i>navigator.appName</i> elle permet de connaître les fonctionnalités que supporte le navigateur de votre visiteur.	5.0 (Windows NT 6.3; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/64.0.3282.18 6 Safari/537.36
<i>language</i>	renvoie une chaîne de caractère donnant la langue utilisée par le navigateur du client. Cette propriété n'est comprise que par les navigateurs supportant les versions 1.2 et supérieures de JavaScript.	fr
<i>mimeTypes</i>	Cette propriété renvoie un tableau répertoriant les types MIME supportés par le navigateur, c'est-à-dire les types de fichiers enregistrés.	application/pdf application/x-google-chrome-pdf application/x-nacl application/x-pnacl application/x-ppapi-widevine-cdm
<i>platform</i>	Cette propriété renvoie une chaîne de caractère indiquant la plateforme sur laquelle le navigateur fonctionne, c'est-à-dire le système d'exploitation du client. Cette propriété n'est comprise que par les navigateurs supportant les versions 1.2 et supérieures de JavaScript.	Win32
<i>plugins</i>	Cette propriété renvoie un tableau contenant la liste des plugins installés sur la machine cliente.	internal-pdf-viewer mhjfbmdgcfjbbpaeojo fhoefgiehjai

		internal-nacl-plugin widevinecdmadapter. dll
userAgent	retourne la chaîne de caractère qui comprend toutes les informations sur le navigateur de client. Les propriétés ci-dessus offrent un moyen pratique de récupérer une partie de cette information.	Mozilla/5.0 (Windows NT 6.3; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/64.0.3282.18 6 Safari/537.36

ii. Les méthodes de l'objet navigator

Les méthodes de l'objet *navigator* permettent d'effectuer certaines opérations concernant le navigateur du client., Il est indispensable de les faire précéder par *navigator*.

Méthode	Description
javaEnabled()	permet de vérifier si le navigateur du client est configuré pour exécuter des applets Java .
plugins.refresh()	Une méthode de la propriété <i>plugin</i> permet de rafraîchir la liste des plugins installés sur le poste client.
preference("preference",valeur)	permet à un script signé de redéfinir les préférences du navigateur. Le script doit ainsi obtenir les privilèges suffisants pour pouvoir effectuer ces actions.
SavePreferences()	permet de sauvegarder les modifications apportées aux préférences du navigateur du client.

Exemple de script

Voici le script permettant de vérifier que les versions des navigateurs sont supérieures à 9 et Microsoft Internet Explorer 9 (ou supérieur):

```
Nom = navigator.appName;
Version = navigator.appVersion;
ns = (Nom == 'Netscape' && Version >= 9 ) ? 1:0
ie= (Nom == 'Microsoft Internet Explorer' && Version >= 9 ) ? 1:0
if (ns) {
// instructions
}
if (ie) {
//les instructions à exécuter
}
```

5. L'Objet image

L'objet Image permet de manipuler les images d'un document, réaliser des effets de rollovers et faire du pré chargement d'images.

Un objet Image peut se créer de 2 manières. La plus évidente est d'utiliser la balise HTML .

Le JavaScript peut aussi créer des images en JavaScript ; l'intérêt est alors de les pré charger dans le cache du navigateur pour les afficher instantanément à l'utilisateur. L'objet Image est généralement utilisé pour s'assurer qu'une image a été chargée, en utilisant l'événement onLoad.

L'accès à une image de la page se fait facilement par le tableau images de document.

Créer un objet Image

Pour créer un objet "Image":

```
nomImage = new Image([largeur, hauteur])
```

Le but d'utiliser un objet image avec le constructeur Image() est de charger une image du réseau et de la décoder avant que l'on ait besoin de l'afficher. Puis, lorsque l'image doit être affichée, la propriété SRC de l'image peut être fixée à la même valeur que celle utilisée ultérieurement. Le résultat sera que l'image sera obtenue de la mémoire cache plutôt que d'être chargée du réseau. Il est possible d'utiliser cette technique pour créer de l'animation ou pour afficher une de plusieurs images suite aux entrées d'un formulaire.

Syntaxe

Pour définir une image, il faut utiliser la syntaxe HTML habituelle:

```
<IMG SRC="URL" ..>
```

Pour créer un objet "Image" en utilisant le constructeur :

```
nomImage = new Image([largeur, hauteur])
```

nomImage est soit le nom d'un nouvel objet ou la propriété d'un objet existant.

largeur et la hauteur sont en pixel.

Exemple

```
monImage = new Image()
monImage.src = "unelImage.gif"
...
document.images[0].src = monImage.src
```

Utilisation dans un document :

Pour utiliser le tableau d'images dans un objet Document:

```
document.images[index]
```

index est un entier représentant une image dans le document ou une chaîne de caractères contenant le nom d'un objet "Image".

Pour obtenir le nombre d'images dans un document, il faut utiliser la propriété "length".

```
document.images.length
```

Propriétés de l'objet Image :

L'objet "Image" possède les propriétés suivantes:

Propriétés	Description
alt	Infobulle de l'image
complete	une valeur booléenneIndicateur de fin de chargement
fileSize	Poids de l'image
height	Hauteur de l'image
id	Identifiant de l'image
name	Nom de l'image
src	Adresse de l'image
width	Largeur de l'image
border	Bordure de l'image

Pour utiliser les propriétés d'un objet "Image":

1. nomImage.nomPropriété
2. document.images[index].nomPropriété
3. nomFormulaire.elements[index].nomPropriété

nomImage est la valeur de l'attribut NAME d'un objet "Image".

nomFormulaire est soit la valeur de l'attribut NAME d'un objet "form" ou un élément dans le tableau de formulaires.

index, lorsque utilisé dans le tableau d'images, est un entier ou une chaîne de caractères représentant un objet "Image". Lorsqu'il est utilisé dans le tableau d'éléments, index est un entier représentant un objet "Image" dans un formulaire.

Le tableau d'images dans un document possède la propriété suivante:

length (représente le nombre d'images dans un document)

Méthode utilisée:

aucune

Exemple de manipulation d'un objet IMAGE (tableau d'images)

```
<HTML><HEAD><TITLE>Exemple</TITLE>
<BODY>
<SCRIPT>
delai = 120 ;
Numero = 0 ;
tImage = new Array() ;
for(i = 0; i < 2; i++) {
tImage[i] = new Image()
tImage[i].src = "png/h" + i + ".png"}
functionanimate() {
    document.animation.src = tImage[Numero].src ;
    Numero++ ;
}
```

```
        if(Numero>= 2)
            Numero = 0 ;
    }
</SCRIPT>
<IMG NAME="animation" SRC="png/h0.png" onLoad="setTimeout('animate()', delai)"
>
</BODY></HTML>
```